

TECHNICAL RESEARCH · 2026

Harness Engineering 2026

让模型能力成为 可靠任务系统

从执行层原理、组件协同与自优化研究，到国内外厂商和开源社区的落地实践

Evidence-first technical survey

证据截至 2026-06-20 · Asia/Shanghai

版本 1.0 · 公开研究资料

| 目录

00	执行摘要：Agent 能力的最小报告单位已经改变	03
01	Harness 的边界：模型之外，任务系统之内	04
02	执行原理：观察、行动、验证如何形成可靠闭环	05
03	生态协同：Skills、MCP、A2A 与运行时各管一层	07
04	前沿研究：Harness 开始成为可测量、可搜索的程序	09
05	失效模式：增加组件不等于增加能力	11
06	落地实践：国际厂商与国内社区怎样取舍	13
07	参考架构：从最小闭环走向持续优化	15
08	研究议程：下一步要解决可迁移性与治理	17
09	术语、常见误区与进一步阅读	18

Agent 能力的最小报告单位已经改变

把一个基础模型接到真实任务上，最终表现由模型、Harness 与任务环境共同决定。2026 年的最新证据已经足以支持一个谨慎结论：**只报模型名，不再足以解释 Agent 能力**。更可复现的单位是 model-harness configuration，也就是模型与执行配置的组合 [S01, S03]。

四个结论

Harness 是执行系统，不是加长版 Prompt。它决定模型看见什么、能调用什么、状态放在哪里、何时验证、失败后如何恢复，以及满足什么条件才能结束。Prompt 只是其中一个组件 [S01, S07]。

执行对齐能产生一阶影响。Claw-SWE-Bench 在固定模型时观察到最高 27.4 个百分点的 Harness 差异；同一 GLM 5.1 配合不同适配器时，Pass@1 从 19.1% 到 73.4%。这些数字来自单篇预印本，不能直接外推，但足以说明执行契约不是评测噪声 [S03]。

组件越多，系统未必越强。SWE-Skills-Bench 的 49 个公开 Skills 中，39 个没有提高通过率，平均增益仅 1.2%；三个 Skill 因版本不匹配使表现最多下降 10%。多 Agent 也有明显成本，Anthropic 报告其研究系统消耗约为普通聊天的 15 倍 Token [S04, S10]。

前沿正在从手工调参转向轨迹驱动优化。Meta-Harness、RHO、Self-Harness 与 HarnessX 分别探索代码搜索、无标签回顾、自我改进和模型协同进化。它们把执行轨迹变成修改 Harness 的反馈，但目前大多是 2026 年预印本，复现、跨域迁移和防止过拟合仍未解决 [S02, S05, S06, S08]。

判断	证据	读法
Harness 影响可与模型选择同量级	固定模型最高差 27.4 pp [S03]	报告完整组合，不把差异都归因给模型
过程知识需要选择与版本管理	39/49 Skills 无通过率增益 [S04]	Skill 是待评测依赖，不是免费能力
自动优化已经出现强信号	多篇预印本报告 4.7 至 21.4 pp 等不同增益 [S05, S08]	看 held-out、回归门和代码可复现性
生产可靠性来自显式状态与验证	进度文件、Git、轨迹、快照和恢复成为共同组件 [S09, S12, S15]	先建设可观察、可恢复的闭环

本文不做 Harness 产品排行榜。厂商内部数字按厂商自报处理，开源仓库只证明公开能力存在，论文结果只在各自任务、预算和评测协议内成立。

Harness 的边界：模型之外，任务系统之内

本文将 Harness 译作「执行框架」，后文保留英文词。它位于基础模型和外部环境之间，把模型输出变成可执行、可检查、可恢复的任务过程。

一个可操作的定义

Harness-Bench 把 Harness 定义为管理上下文、工具、状态、约束、权限、追踪和恢复的系统层；Natural-Language Agent Harnesses 又补上验证、停止条件、预算与多次 Agent 调用的组织方式 [S01, S07]。两者的公共部分可以写成一个工程定义：**Harness** 是围绕固定模型的非参数执行系统，它编排观察、动作与反馈，同时约束动作能影响的环境。

这个定义把三个常被混用的对象分开。模型是从上下文到输出的学习函数；Agent 是一个或多个模型调用与外部动作组成的执行过程；Harness 则规定这个过程怎样运行。没有外部动作的一次文本生成可以使用 Prompt，却通常不需要完整 Harness。只要任务涉及文件、浏览器、数据库、长程状态或其他 Agent，执行层就开始决定结果。

边界内外

对象	是否属于 Harness	原因
系统指令、AGENTS.md、任务契约	是	控制模型如何解释任务与完成条件
工具定义、MCP 连接、文件编辑器	是	规定动作空间与观察格式
Session、Memory、工作区与检查点	是	保存跨步骤和跨上下文状态
编排、重试、预算、审批和停止规则	是	决定执行路径与风险边界
基础模型权重与训练语料	否	属于参数能力，Harness 不修改权重
只负责展示的聊天 UI	通常否	除非它承载审批、恢复或状态协议
任务评测容器	相邻层	用于固定外部条件，不应与被测 Harness 混为一体

为什么要按组合报告能力

传统模型评测试图固定提示和推理条件，Agent 评测却无法轻易固定执行层。工具接口、补丁提取、上下文压缩、超时和停止规则会改变模型实际收到的信息，也会改变最终产物是否进入评分器。Claw-SWE-Bench 的完整适配器把通用 Agent 的工作区状态转换为 SWE-bench 接受的补丁契约，最小适配

器则没有完成同样的对齐。19.1% 与 73.4% 的差距首先说明适配器是否完成了任务契约，不应被解释成某个孤立技巧带来 54.3 个百分点增益 [S03]。

因此，公开结果至少应同时披露模型版本、Harness 版本、任务提示、工具集、预算与超时、并发方式、环境镜像、补丁或产物提取规则、缓存口径和评测器。Harness-Bench 进一步建议保留执行轨迹与验证器输出，使完成率、过程质量、效率、安全和可审计性能够一起分析 [S01]。

最小报告单位

任务结果 = $f(\text{模型}, \text{Harness}, \text{环境}, \text{预算}, \text{评测器})$ 。如果其中一项没有公开，排行榜差异就可能来自未披露的执行条件。

02 · EXECUTION MECHANICS

观察、行动、验证如何形成可靠闭环

可靠 Harness 不靠一条万能指令。它把一次开放式推理改造成一组有状态、可验证的状态转移，并让失败信息能够回到下一次决策。

核心循环

```
while budget remains:
    observation = assemble(task, state, evidence, tool_results)
    action      = model(observation)
    effect      = execute(action, permission_policy, sandbox)
    verdict     = verify(effect, acceptance_contract)
    state       = checkpoint(state, action, effect, verdict)
    if verdict.complete: return artifact
    if verdict.recoverable: adapt_or_retry()
return explicit_failure
```

每次模型调用都要拿到与当前环境一致的观察，动作结果也要由确定性证据验证。Harness-Bench 把一种常见失败称为执行对齐失效(execution-alignment failure)：推理听起来合理，但已经脱离工具反馈、工作区状态、证据或产物契约 [S01]。

八层组件栈



图 1 | Harness 的层次不等于固定实现。可靠性来自跨层契约一致，尤其是任务产物、工具反馈、工作区状态和验证器使用同一份事实。

五种增益机制

信息对齐，让模型在恰当时刻获得足够而不过量的任务信息。OpenAI 的案例把短 AGENTS.md 当目录，把细节放入可检索、可验证的仓库文档；Anthropic 则用进度文件和 Git 历史跨越上下文窗口 [S09, S13]。

动作可供性，让工具的名字、参数、错误和输出符合模型能稳定使用的形式。Anthropic 的工具测试 Agent 通过反复调用并重写描述，使后续任务完成时间下降 40%。这是厂商内部结果，但它说明工具接口本身可以成为性能变量 [S10, S11]。

状态连续性，把长程工作从聊天记录迁到持久化 Session、文件、事件日志和检查点。Managed Agents 把 Session、Harness 与 Sandbox 分开，OpenAI Agents SDK 也把 Harness 状态放到可快照、可重建的计算环境之外 [S12, S15]。

反馈闭合，用测试、浏览器、数据校验或人工审批证明动作达成任务契约。Google ADK 同时评估工具轨迹和最终响应；OpenAI 的 Agent-first 仓库把架构约束写成 Lint 和结构测试，并让错误消息直接给 Agent 修复指引 [S13, S17]。

错误围堵，用最小权限、沙箱、超时、幂等重试和回滚限制失败传播。一次错误需要留下可诊断证据，并且不能污染下一次运行。

Skills、MCP、A2A 与运行时各管一层

Harness 生态的价值不在于把所有能力装进一个框架，而在于让过程知识、工具、Agent 协作、执行状态和计算环境拥有清楚的接口。协议解决可连接性，Harness 负责把连接变成可控任务。

四种接口不要混为一谈



图 2 | 互操作标准只定义边界和消息。选择何时加载 Skill、调用哪一个工具、是否并发、怎样审批以及如何验证，仍是 Harness 的职责。

Agent Skills：把过程知识变成可版本化依赖

Agent Skills 规范用一个必需的 SKILL.md 和可选的脚本、参考资料、模板目录封装过程知识。发现阶段只加载名称与描述，任务匹配后再读取完整指令，需要时才打开附属资源，这种渐进披露降低了常驻上下文成本 [S21]。它适合承载代码审查流程、文档生产规范和领域操作手册，却不适合承担精确的权限检查或数据验证。

SWE-Skills-Bench 的负面结果说明，Skill 必须像依赖一样选择、锁版本和回归测试。通用建议往往重复模型已有能力，过时建议会与仓库当前状态冲突。Skill 的价值来自窄而准的程序性知识，以及可执行的验收条件，不来自字数 [S04]。

MCP：标准化工具与上下文，不替代工具设计

MCP 2025-11-25 定义 Resources、Prompts 和 Tools，并要求主机在传输资源和调用工具前获得用户同意。规范同时承认，协议本身无法强制所有安全原则 [S19]。截至 2026-06-20，下一版仍是

2026-07-28 Release Candidate, 方向包括无状态协议内核、独立扩展、长任务和更严格的授权模型, 不能写成已经发布的稳定规范 [S20]。

把一个糟糕工具接入 MCP, 只会标准化糟糕接口。工具需要互斥而清楚的职责、有限参数、结构化错误、紧凑输出和可评测描述。工具数量很大时, 应先检索工具元数据, 再按任务暴露小集合, 避免让描述挤占任务上下文 [S10, S11]。

A2A 与 App Protocol: 一个管 Agent 间任务, 一个管客户端体验

A2A 1.0 用 AgentCard 描述能力, 用 Task、Message、Artifact 和扩展表达异步协作, 允许独立且内部不透明的 Agent 交换任务和产物 [S22]。它适合跨团队、跨厂商委派, 不负责暴露对方内部 Memory 或工具。Codex App Server 则面向产品客户端, 用双向事件流表达 thread、turn、item、diff 和审批, 目标是让 CLI、IDE、桌面和 Web 复用同一 Harness [S14]。两者解决的问题不同。

一个端到端例子

设想一个需要修复支付回调并发布审计报告的工程任务。Harness 先从仓库 AGENTS.md 读取边界, 再按任务加载支付 Skill。它通过 MCP 获取日志和工单, 只把只读工具交给诊断 Agent; 修复 Agent 在隔离工作区修改代码, 测试 Agent 使用独立上下文验证。若合规审查属于另一套企业系统, Harness 通过 A2A 委派并接收签名 Artifact。客户端从 App Protocol 看到进度和高风险审批, 最终只有测试、审计与产物校验都通过, 任务才进入完成状态。

判断生态协同质量, 要看每个接口是否减少了隐式状态、重复适配和权限歧义, 接入协议的数量没有解释力。

Harness 开始成为可测量、可搜索的程序

2026 年的研究出现一条清楚路线：先把 Harness 作为独立变量测量，再把策略从控制器代码中抽出，最后用轨迹和回归测试自动修改它。

第一步是把 Harness 从排行榜噪声中分离

Harness-Bench 固定任务环境、预算、超时和评测器，同时保留不同 Harness 的原生执行行为。它包含 106 个离线沙箱任务和 5,194 条轨迹，覆盖软件工程、数据分析、知识工作与权限敏感流程 [S01]。这种设计提高了现实性，却只能比较完整配置，无法证明某一条重试规则或上下文策略单独造成差异。

Claw-SWE-Bench 进一步用统一适配器契约比较通用 Agent Harness。350 个任务跨 8 种语言和 43 个仓库，并同时报告 Pass@1、成本、时长和缓存。固定模型时 Harness 差异最高 27.4 个百分点，固定 OpenClaw 时模型差异为 29.4 个百分点 [S03]。排名本身解释力有限，结果证明模型与 Harness 都属于一阶变量。

八项前沿工作放在同一张表里

工作	优化对象	反馈	关键结果	主要限制
Harness-Bench [S01]	完整配置	验证器与轨迹	106 任务, 5,194 轨迹	诊断, 不拆单组件因果
Claw-SWE-Bench [S03]	Harness 与适配器	补丁评测与成本	固定模型最高差 27.4 pp	集中在编码任务
NLAH [S07]	自然语言策略	模块行为与任务结果	跨编码、终端、计算机使用可执行	确定性机制仍需代码
Meta-Harness [S05]	Harness 代码	全历史代码、分数、轨迹	分类 +7.7 pp, 4 倍少 Token	外层搜索成本高
RHO [S06]	工具、Skills、工作流	自验证、自一致性、自偏好	SWE-Bench Pro 59% → 78%	没有独立真值反馈
Self-Harness [S08]	模型自己的 Harness	失败聚类与回归测试	3 个模型 held-out +14.2 至 +21.4 pp	只测 Terminal-Bench-2.0
HarnessX [S02]	类型化组件与模型	轨迹、Critic、RL 信号	15 个配置平均 +14.5%	代码截至证据日待发布
SWE-Skills-Bench [S04]	单个 Skill	成对执行测试	平均 +1.2%, 39/49 无增益	初步预印本结果

策略与机制分离，让 Harness 可以消融

Natural-Language Agent Harnesses 把任务阶段、角色、状态规则、验证策略和停止条件写成自然语言策略，由共享运行时解释；测试、解析器、沙箱和产物校验仍保留在确定性代码中 [S07]。这个分工比「用自然语言替代代码」更实际。自然语言适合表达可读、可修改的政策，代码负责精确执行和可重复验证。

可消融性是关键收益。Verifier、上下文压缩、文件状态、多候选搜索和恢复策略如果拥有清楚边界，就能单独关闭并观察结果。没有模块边界的 Harness 只能整体比较，研究者无法知道增益来自哪一层，也很难把有效机制迁到另一套运行时。

轨迹驱动自优化循环

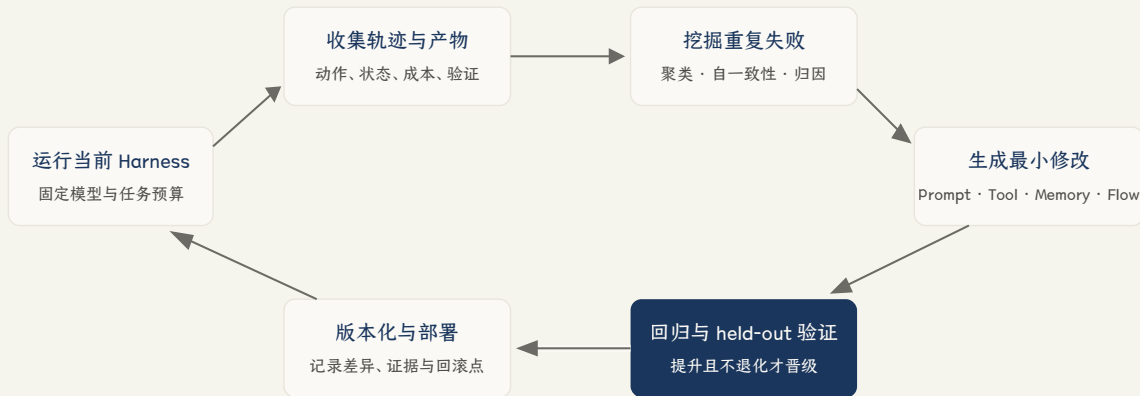


图 3 | 自动优化的可信度取决于最右下角的回归门。没有 held-out 任务、最小修改和回滚点，自我改进很容易变成对当前评测器的过拟合。

Meta-Harness 让外层 Agent 通过文件系统访问所有历史候选的代码、轨迹和分数，避免把反馈压成一个标量。它在在线分类中比强基线高 7.7 个点，同时使用 4 倍更少的上下文 Token；发现的数学推理 Harness 在 5 个未见模型上平均提升 4.7 个点 [S05]。

RHO 不使用标注验证集，而从历史任务选出困难且多样的核心集，进行多次 Rollout，再用自验证、自一致性和成对自偏好选择修改。论文报告单轮把 SWE-Bench Pro 从 59% 提升到 78% [S06]。Self-Harness 则让同一个模型分析自己的失败，提出小改动，并只接纳通过回归测试的版本；三个模型在 held-out 任务上分别提高 21.4、14.3 和 14.2 个百分点 [S08]。

HarnessX 把上下文、工具、Skills、控制流和 Memory 做成类型化原语，再用 Digester、Planner、Evolver、Critic 四阶段管线修改它们，并尝试把轨迹同时用于 Harness 更新和模型 RL。论文报告 15 个配置平均提升 14.5%，最高 44.0%，但截至证据日完整代码尚未发布，应把它看成研究方向而非可复现事实 [S02]。

增加组件不等于增加能力

Harness 的每一层既可能修复模型弱点，也可能引入新的失配。设计质量取决于能否用轨迹和回归测试证明边际价值，而不是组件数量。

上下文越多，相关信息占比可能越低

大而全的系统指令会挤占任务、代码和证据的空间，陈旧规则还会与当前仓库冲突。OpenAI 的案例从大型 AGENTS.md 退回到约 100 行的入口地图，把细节放入结构化文档并用 CI 检查交叉链接和新鲜度 [S13]。Agent Skills 的渐进披露也采用同一原则，只在发现、激活和资源读取三个阶段逐步加载 [S21]。

压缩同样有风险。摘要可能丢掉未完成动作、错误根因和任务契约。长程 Harness 需要把关键状态外置到结构化文件、版本历史或事件日志，而不是相信下一次压缩仍保留全部决定 [S09, S12]。

工具数量会制造选择错误

工具越多，描述占用的上下文越大，命名重叠也会增加错误路由。一个「搜索」工具如果同时承担网页、Slack、数据库和仓库检索，模型难以从名称预测成本与适用边界。更稳的做法是命名空间、窄接口、结构化错误和按任务检索工具。MCP 解决发现与调用格式，却不保证工具描述清楚，也不保证服务端实现安全 [S11, S19]。

Skill 可能重复常识，也可能携带过期事实

SWE-Skills-Bench 为每个任务建立有无 Skill 的成对实验。49 个 Skills 中只有 7 个产生明显增益，39 个没有提高通过率；Token 开销最高增加 451% 而通过率不变，三个 Skill 因版本不匹配最多使通过率下降 10% [S04]。这组初步结果不应被外推为「Skills 无用」，它支持更窄的判断：只有能补足特定知识缺口、与当前版本一致并通过任务回归的 Skill 才值得常驻。

多 Agent 适合宽度搜索，不适合所有依赖图

Anthropic 的多 Agent 研究系统在内部评测中比单 Agent 高 90.2%，Token 消耗约为普通聊天的 15 倍。其分析中，Token 用量解释 BrowseComp 评测 80% 的表现方差；收益集中在可以并行探索、信息超过单上下文窗口且任务价值覆盖成本的场景 [S10]。编码任务中共享状态多、依赖强，过早拆分会带来重复搜索、合并冲突和协调等待。

委派必须包含目标、输出格式、证据要求、工具范围和结束条件。Subagent 应拥有隔离上下文和最小工具集，返回压缩后的产物而不是全部对话。主 Agent 负责解决冲突与验收，不能把多个未经核验的答案直接拼接。

自动优化会把评测器弱点学进 Harness

自动 Harness 搜索面临与强化学习相似的问题：奖励投机、灾难性遗忘、探索不足和跨任务干扰。HarnessX 用 Critic、确定性门和任务变体隔离处理这些风险；Self-Harness 要求候选修改小而具体，并用 held-out 回归决定是否晋级 [S02, S08]。如果修改同时改了任务提示、预算和评测器，增益就无法归因。

RHO 用自偏好替代外部标注，降低数据门槛，也让判断者与执行者可能共享偏差。生产系统可以用它生成候选，最终接纳仍应依赖确定性验证、独立模型或人工抽检，特别是安全、财务和合规任务 [S06]。

七个可观察的坏味道

症状	常见根因	应记录的证据	首个修复动作
合理解释，错误产物	推理脱离工作区状态	最后一次读取与实际文件哈希	最终回答前重开产物并运行验证器
重复调用同一失败工具	错误语义不可判别	工具名、参数、错误码、序列号	增加结构化错误与重试上限
长任务提前宣布完成	完成条件只存在于自然语言	需求矩阵与未通过项	把验收条件写成不可删除的状态文件
上下文越长表现越差	入口文档过大或规则过期	实际加载片段和命中来源	入口改成索引，按需读取
并发后结果重复或冲突	任务边界和产物契约不清	委派包、返回格式、合并记录	缩小并发面，明确唯一责任
Benchmark 提升，线上退化	过拟合评测器或任务分布	held-out、时间切分、失败簇	冻结评测器，增加回归与回滚门
重启后无法继续	状态留在进程或 Sandbox	Session 事件、检查点、产物清单	外置状态并验证重建流程

国际厂商与国内社区怎样取舍

优秀实践并没有收敛到一个框架。它们收敛的是工程问题：知识怎样被 Agent 找到，动作怎样被验证，状态怎样跨越运行时，权限怎样约束，轨迹怎样进入下一轮改进。

OpenAI：把仓库、运行时和任务板都改造成控制面

OpenAI 报告一个内部产品在 5 个月内由 3 名工程师驱动 Codex 合并约 1,500 个 PR，代码约 100 万行，估计开发时间为手写的十分之一；所有数字都是厂商案例，没有独立复测 [S13]。真正可迁移的机制有四个：仓库知识成为唯一事实源；应用 UI、日志、指标和 Trace 对 Agent 可读；架构约束进入 Lint 和结构测试；人类把意图与验收写清，Agent 执行与自评。

Codex App Server 把同一 Agent Loop 暴露给 CLI、IDE、桌面和 Web。Thread 保存持久会话，Turn 表示一次用户触发的工作，Item 表示消息、工具、审批与 Diff 等原子事件。双向 JSON-RPC 让服务器可以请求审批并暂停执行 [S14]。这套接口展示了 Harness 产品化的一项关键分离：UI 不再复制 Agent Loop，而是订阅稳定事件。

Agents SDK 继续把 Harness 与计算环境拆开。Harness 保存状态和调度，Sandbox 承担文件、Shell 与依赖执行，Manifest 描述工作区，快照与重建让容器失效后仍能继续 [S15]。Symphony 则把 Linear 一类任务板变成状态机，每个未阻塞 Issue 对应独立工作区和 Agent。OpenAI 报告部分团队前三周落地 PR 数增长 500%，这同样属于厂商自报，适合观察 workflow 形态，不适合作为普遍生产率预测 [S16]。

Anthropic：长程状态、工具可用性和并行搜索分开优化

Anthropic 的长程实验把第一轮初始化与后续增量执行分开。初始化器创建环境、需求矩阵、进度文件和 Git 基线；后续 Agent 一次完成一个功能，运行端到端验证，提交清楚的状态给下一轮 [S09]。这套方法没有复杂编排器，却抓住了长程任务的核心：新上下文必须能从持久化产物恢复事实，而不是从模糊摘要猜测。

Managed Agents 把 Session 视为追加式事件日志，把 Harness 视为调用模型与路由工具的循环，把 Sandbox 视为执行环境。三者用稳定接口连接，允许模型升级时替换 Harness 假设，也允许长任务在计算环境迁移后继续 [S12]。Claude Agent SDK 进一步把文件系统、Shell、编辑和搜索作为通用计算机接口，其基本反馈循环是收集上下文、行动、验证、重复 [S18]。

多 Agent 研究系统使用编排者-工作者结构，主 Agent 规划并行搜索，Subagent 用独立上下文压缩局部证据，Citation Agent 再定位引用。团队同时限制 Agent 数和工具调用预算，避免简单问题生成 50 个 Subagent [S10]。这个案例说明并发策略应由任务可分解性驱动，而不是产品宣传中的「Agent Team」标签。

Google 与社区框架：把状态机、评测和人类介入做成基础设施

Google ADK 区分最终响应评测与工具轨迹评测，并支持基于真值或 Rubric 的工具使用指标。开发期可以把单个 Session 当单元测试，生产回放则检查真实动作序列 [S17]。LangGraph 强调持久执行、短期与长期 Memory、人类介入和状态转移追踪；OpenHands 把可组合 Agent SDK 与 Docker Sandbox 分开 [S23, S24]。这些框架的共同价值在基础设施，不在某个固定 Prompt。

国内大厂与社区：从模型适配走向完整 Harness

项目	核心形态	值得借鉴	公开边界
ByteDance DeerFlow [S25]	长程 Super Agent Harness	Skills 渐进加载、Subagent 隔离、Memory、Sandbox、Gateway 与产物目录	Gateway 运行状态仍在进程内，公开文档要求单 Worker，横向状态桥尚未提供
ByteDance Trae Agent [S26]	研究友好的编码 Agent	多模型配置、文件与 Shell 工具、完整轨迹 JSON、容器执行	模块透明度高，生产编排与多租户不是核心目标
Alibaba Qwen-Agent [S27]	围绕 Qwen 的 Agent 组件库	模型特定 Function Calling 解析、MCP、RAG、Docker Code Interpreter	不同 Qwen 版本需要不同工具解析配置，体现 Harness 与模型耦合
AgentScope [S28]	Agent-Oriented 生产框架	细粒度权限、多租户、多 Session、Workspace/Sandbox、团队与后台任务	能力面广，具体任务仍需自己建立评测和策略

DeerFlow 2.0 的公开架构最接近本文所说的完整 Harness：Skills 与工具提供能力，Subagent 承担隔离协作，文件系统保存工作状态，Sandbox 隔离执行，Memory 跨 Session 保留信息，Gateway 管理流式事件与运行状态 [S25]。它也公开写出当前限制：Gateway 默认单 Worker，直接提高 Worker 数会破坏取消、SSE 重连和去重。这种边界披露比单纯罗列功能更有工程价值。

Trae Agent 把每次模型交互、工具动作和执行元数据写入轨迹文件，适合做 Harness 消融和故障分析 [S26]。Qwen-Agent 针对 Qwen 系列的工具调用格式和推理字段做模型适配，并提供 MCP、RAG 和容器代码解释器 [S27]。AgentScope 把权限、多租户、多 Session 与工作区隔离放到框架层，代表从研究编排走向服务治理的另一条路线 [S28]。

07 · REFERENCE ARCHITECTURE

从最小闭环走向持续优化

生产 Harness 不应从多 Agent 拓扑开始。最小可用版本只有一条路径：一个明确任务契约，一组必要工具，一个持久状态载体，一个独立验证器，以及失败时可恢复的执行环境。

最小可用 Harness

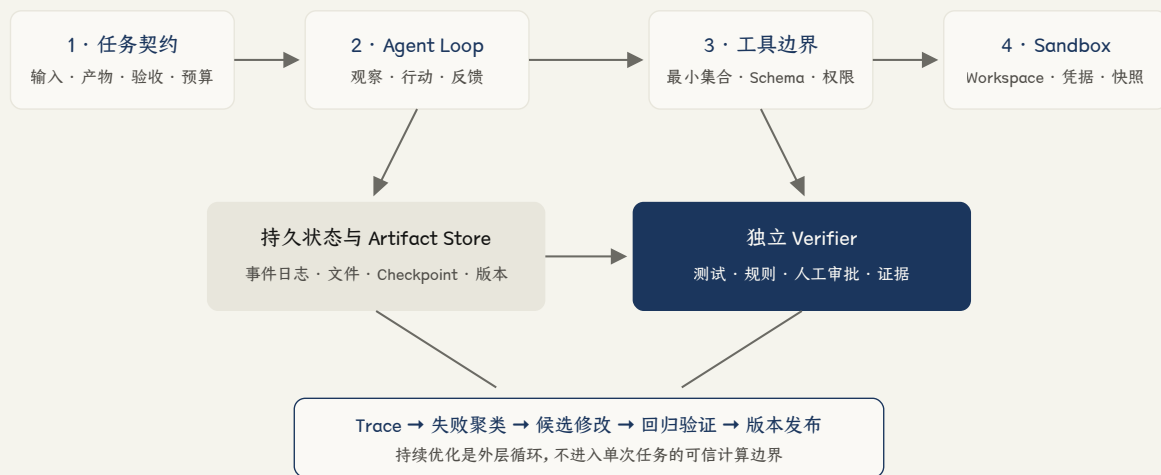


图 4 | 任务执行与 Harness 优化分成两个循环。单次任务只运行已发布版本，优化器在外层读取脱敏轨迹并通过回归门发布新版本。

任务契约先于 Prompt

契约至少写清输入、输出 Artifact、允许的动作、完成条件、预算和失败格式。编码任务的 Artifact 是可应用补丁及测试证据；研究任务的 Artifact 是带来源定位的报告及证据包；数据任务的 Artifact 是可解析文件、Schema 和校验结果。完成条件必须由验证器读取 Artifact 判断，不能依赖 Agent 自述。

状态分成四类

状态	寿命	载体	禁止做法
Turn 状态	一次模型调用	当前观察与工具结果	把全部历史无差别塞回上下文
Task 状态	一次任务	事件日志、工作区、任务清单	只保存在进程内存
User/Team Memory	跨任务	结构化事实、偏好、授权范围	把推断写成事实，永久保存敏感内容
Harness 经验	跨版本	脱敏轨迹、失败簇、回归集	直接把单次成功轨迹升级为规则

Verifier 与执行 Agent 保持不同职责

Verifier 应尽量依赖确定性信号，比如测试退出码、文件 Schema、数据库约束、页面选择器和来源定位。需要语义判断时，再使用独立 Rubric、不同模型或人工抽检。执行 Agent 不应修改测试、评分器和不可变需求；如果业务允许更新验收标准，这必须是单独审批事件。

五级建设顺序

级别	独立可交付能力	进入下一级的门
L1 · 可执行	单 Agent、最小工具、明确 Artifact	核心任务有确定性通过/失败信号
L2 · 可诊断	结构化事件、工具错误、成本和产物追踪	失败能归到具体状态转移
L3 · 可恢复	持久 Session、Checkpoint、幂等重试、回滚	杀掉进程后能从检查点继续
L4 · 可协作	按需 Subagent、隔离工具、结构化 Handoff	并发在成本约束下显著优于单 Agent
L5 · 可优化	失败聚类、候选 Harness、held-out 回归、版本发布	跨时间与任务分布保持增益，无关键退化

每一级都能独立上线。没有必要为了使用 A2A 或多 Agent 跳过 L2 和 L3。无法诊断和恢复的并发系统，只会更快地产生无法解释的失败。

评测矩阵

离线评测同时覆盖结果、过程、成本、安全和恢复。结果看 Artifact 是否满足契约；过程看工具选择、无效循环和证据链；成本看 Token、时长、工具与 Sandbox 费用；安全看越权、注入和数据外传；恢复看超时、工具故障、容器丢失和断网后的状态。上线后按失败簇采样真实轨迹，定期加入冻结回归集。

一周内可以完成的最小动作

选择 20 个真实任务，冻结模型、工具和预算；为每个任务写 Artifact 验收器；记录完整轨迹；只修改一个 Harness 组件；用成对结果决定保留或回滚。

| 下一步要解决可迁移性与治理

前沿结果证明 Harness 可以被优化，却没有证明它能稳定跨模型、跨任务和跨组织迁移。真正困难的问题从「能不能改进」转向「改进是否可归因、可复现、可治理」。

从完整配置比较走向组件因果

Harness-Bench 和 Claw-SWE-Bench 已经把 Harness 变成评测轴，但完整配置中 Prompt、工具、状态和恢复相互耦合。下一步需要类型化生命周期 Hook、统一事件 Schema 和严格消融，使研究者能回答「哪一类任务受益于哪一层修改」。NLAH 与 HarnessX 的模块化尝试提供了表达方法，仍需要跨运行时复现 [S01, S02, S07]。

模型特定优化与跨模型迁移要同时报告

Self-Harness 发现不同模型需要不同改动：有的模型需要提前产出文件，有的需要停止重复命令，有的需要保存 Shell 环境 [S08]。这说明「最佳 Harness」很可能不存在。研究应同时给出同模型 held-out、跨模型迁移和新模型升级后的退化，区分通用机制与模型补丁。

自动优化需要安全发布系统

未来的 Harness Optimizer 更像配置编译器与发布系统，而不是一次 Prompt 改写。候选版本要经过静态检查、权限差异审查、成本上限、冻结回归、Canary 和自动回滚。高风险工具的权限策略不能由同一个优化器在没有独立审批的情况下放宽。轨迹可能包含用户数据、凭据和内部代码，训练与回放前需要最小化、脱敏和保留期限。

Harness 与模型会形成双向训练循环

HarnessX 提出把 Harness 进化产生的轨迹作为模型 RL 信号，再让更新后的模型参与下一轮 Harness 进化 [S02]。这条路线可能减少运行时补丁，也可能把 Harness 的局部偏差固化进模型。评测必须保留一组不参与两侧优化的外部任务，并追踪模型升级后哪些 Harness 规则已经过时。

协议层会更轻，应用状态仍然很重

MCP 2026 Release Candidate 把协议内核改为无状态，并把长任务与 UI 放入扩展，目标是更容易部署在普通 HTTP 基础设施上 [S20]。无状态协议不代表无状态 Agent。Session、Memory、审批、Artifact 和恢复仍由 Harness 或应用保存。A2A 也刻意不要求共享对方内部状态 [S22]。未来架构会更重视状态所有权和可迁移格式。

Agent 可读性会成为组织基础设施

OpenAI 的案例把仓库内知识、应用 UI、日志、指标与架构约束都改成 Agent 可检索和可验证的表面 [S13]。这类「Agent legibility」要求事实源有索引、有所有者、有新鲜度检查，并能通过工具读取，文档数量本身没有价值。它同时改善新工程师上手和自动化质量，也会增加维护与治理成本。

成本、延迟与风险要进入同一条 Pareto 前沿

只追求通过率会奖励更长上下文、更多 Rollout 和更多 Subagent。Claw-SWE-Bench 已经同时报告成本，Anthropic 的多 Agent 案例也公开了 15 倍 Token 量级 [S03, S10]。下一代基准应统一披露缓存、并发、重试、Sandbox 和外部工具费用，并报告高风险动作数和人工审批时间。

研究问题	需要的最小证据	不能接受的替代
一个组件是否有效	固定其他变量的成对消融	比较两套完全不同的产品
增益是否迁移	新任务、新模型和时间切分	在优化集上重复测试
自动修改是否安全	权限 Diff、冻结回归、Canary、回滚	由优化器自评后直接全量发布
多 Agent 是否值得	相同预算下的质量-成本前沿	只报最高成功率
协议是否提升可靠性	故障注入、兼容性和授权测试	统计接入数量

09 · REFERENCE

术语、常见误区与进一步阅读

术语表

术语	本文定义
Harness	围绕固定模型的非参数执行系统，管理上下文、工具、状态、控制流、权限、追踪、恢复与评测。
Agent	一个或多个模型调用与外部动作组成的执行过程。
Agent Loop	观察、决策、动作、反馈、验证和停止组成的循环。
Trajectory / Trace	任务运行中的模型调用、工具动作、状态变化、成本和验证事件序列。
Artifact	任务交付物，如补丁、文件、报告、数据表、审批记录或结构化结果。
Agent-Computer Interface	Agent 与计算环境交互的工具、观察和错误接口。
Skill	按需加载的过程知识包，可包含指令、脚本、参考资料和模板。
MCP	连接 LLM 应用与 Resources、Prompts、Tools 的开放协议。
A2A	独立 Agent 系统间交换能力描述、任务、消息和 Artifact 的协议。
Verifier	根据任务契约检查过程或 Artifact 的确定性规则、模型或人工角色。
Harness Optimization	基于评测与轨迹修改 Prompt、工具、Memory、控制流或运行时策略的外层过程。

常见误区

把 Harness 当 Prompt 模板。Prompt 无法独自提供持久状态、工具执行、权限、恢复和验证。先画执行状态，再写指令。

先上多 Agent，再补评测。没有单 Agent 基线和任务可分解性证据，多 Agent 的增益与成本都无法归因。先测一条闭环。

把 MCP 接入数当能力。协议只让工具可发现和可调用。工具质量、授权、输出大小和故障语义仍需单独评测。

让执行 Agent 同时改测试。Agent 可以通过降低标准制造成功。测试和不可变验收条件需要权限隔离或独立审批。

用更多上下文修复遗忘。长上下文会引入噪声和陈旧规则。关键状态写入结构化 Artifact，其余按需检索。

把厂商数字直接横比。任务、预算、模型版本、缓存和适配器不同。厂商案例用于理解机制，不用于排行榜。

自动优化后直接发布。自偏好和 LLM Judge 可能共享偏差。候选版本必须经过 held-out 回归、权限 Diff 和回滚演练。

Further Reading

1. 最佳起点：Harness-Bench [S01]，给出可操作定义、评测单位和 106 个可复现实务任务。
2. 最佳系统设计材料：OpenAI Harness Engineering [S13]，展示 Agent 可读仓库、验证器、可观测性和架构约束怎样协同。
3. 最佳长程实践：Anthropic Effective Harnesses [S09]，用初始化器、进度文件、Git 和端到端测试解决跨上下文连续性。
4. 最佳自动优化论文：Meta-Harness [S05]，解释为什么完整代码、轨迹和历史比标量反馈更适合 Harness 搜索。
5. 最佳反例：SWE-Skills-Bench [S04]，提醒读者过程知识的价值取决于任务匹配、版本和回归验证。

研究方法与限制

资料检索截至 2026-06-20，优先使用论文原文、正式规范、官方工程文章与官方开源仓库。2026 年 Harness 研究大多仍是预印本，本文按前沿信号处理；论文增益只在各自任务和预算内成立。厂商内部评测与生产率数据标为 C/D 级证据，不当作独立验证。开源项目按截至证据日的仓库提交快照描述公开能力，未运行统一模型与任务横评。

本文没有覆盖所有 Agent 框架，也没有把 UI 产品、纯 Prompt 库和基础模型训练框架纳入主比较。金融、医疗、法律和高权限自动化需要额外的监管、审计和人类授权设计，本文的参考架构不能替代领域控制。

参考资料

- [S01] Yao et al. Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows. 2026-05-27.
- [S02] Chen et al. HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry. 2026-06-12.
- [S03] Zheng et al. Claw-SWE-Bench. 2026-06-10.
- [S04] Han et al. SWE-Skills-Bench. 2026-03-16.
- [S05] Lee et al. Meta-Harness: End-to-End Optimization of Model Harnesses. 2026-03-30.
- [S06] Pan et al. Retrospective Harness Optimization. 2026-06-04.
- [S07] Pan et al. Natural-Language Agent Harnesses. 2026-03-26.
- [S08] Zhang et al. Self-Harness: Harnesses That Improve Themselves. 2026-06-08.
- [S09] Anthropic. Effective harnesses for long-running agents. 2025-11-26.
- [S10] Anthropic. How we built our multi-agent research system. 2025-06-13.
- [S11] Anthropic. Writing effective tools for agents. 2025-09-11.
- [S12] Anthropic. Scaling Managed Agents. 2026-04-08.
- [S13] OpenAI. Harness engineering: leveraging Codex in an agent-first world. 2026-02-11.
- [S14] OpenAI. Unlocking the Codex harness: how we built the App Server. 2026-02-04.
- [S15] OpenAI. The next evolution of the Agents SDK. 2026-04-15.
- [S16] OpenAI. An open-source spec for Codex orchestration: Symphony. 2026-04-27.
- [S17] Google. ADK: Why evaluate agents. Checked 2026-06-20.
- [S18] Anthropic. Building agents with the Claude Agent SDK. 2025-09-29.
- [S19] Model Context Protocol. Specification 2025-11-25.
- [S20] Model Context Protocol. 2026-07-28 Specification Release Candidate. 2026-05-21.
- [S21] Agent Skills. Specification and documentation. Snapshot 2026-05-20.
- [S22] Linux Foundation. A2A Protocol Specification 1.0. 2026.
- [S23] LangGraph. Repository snapshot. 2026-06-19.
- [S24] OpenHands. Repository snapshot. 2026-06-19.
- [S25] ByteDance. DeerFlow snapshot. 2026-06-20.
- [S26] ByteDance. Trae Agent snapshot. 2026-02-05.
- [S27] Alibaba. Qwen-Agent snapshot. 2026-03-04.
- [S28] AgentScope. Repository snapshot. 2026-06-19.